Ders 20 Çalışma Özeti

Ders 20 Çalışma Özeti

Genel Konular

- Dersin tarihi 30 Kasım 2021 Salı. Ders Zoom üzerinden yapıldı.
  - Bu kayıt 3. uygulama (laboratuvar) oturumudur. Dersin hocası oturumu açıp uygulamayı yürüten asistana devrediyor; asistanın Zoom etiketi "betül turkkol".
  - Asıl anlatım yaklaşık 00:13'te başlar.
  - Oturum yaklaşık 00:38-00:47 arasında bağlantı sorunu nedeniyle kesildi. 01:12'den sonra ekran paylaşımı tamamen çöktü ve uygulama yarım kaldı.
  - Kaydın ikinci video parçası sessizdir.
- Süreç (process) kavramının tekrarı
  - Süreç tablosu (process table) ve süreç kontrol bloğu (process control block).
- Süreç oluşturma: `fork()` ve `exec()` ailesi
  - Dört `fork` örneği ve bir `execl` örneği.
- Süreç sonlandırma: `exit()`, `atexit()`, `wait()`
- Zombi (zombie) ve öksüz (orphan) süreçler
- Süreçler arası iletişim (interprocess communication, IPC)
  - Borular (pipe) ve adlandırılmış borular (named pipe, FIFO).
  - Basit bir boru örneği ve dosya adı gönderen istemci-sunucu boru örneği.
- Anlatılamayan kısımlar: FIFO örnekleri ve sinyaller (signal, `sigaction`).

Hocanın Özellikle Vurguladığı Kısımlar

- `fork()` ebeveynde (parent) çocuğun (child) PID'sini, çocukta 0 döndürür.
  - Çocuk kendi PID'sini `getpid()` ile alır.
- `fork()` sonrası çocuk, kodun `fork()`'tan sonraki kısmını çalıştırır. `fork()`'tan önceki satırlar yalnızca ebeveynde bir kez çalışır.
- Ebeveyn ve çocuğun hangi sırayla çalışacağına işletim sistemi karar verir. Sırayı garanti etmek için `wait()` kullanılır.
- Genelde süreci birebir kopyalamak değil, ebeveyn ile çocuğun farklı işler yapması istenir. Bunun için `if-else` ile ayrım ya da `exec` kullanılır.
- Boru tek yönlüdür. Kullanılmayan uç baştan kapatılmalıdır.
  - Yazan taraf okuma ucunu, okuyan taraf yazma ucunu kapatır.
- Uygulamanın kalanı için telafi yapılacak.
  - Borular baştan alınacak; FIFO ve sinyal örnekleri anlatılacak.
  - Telafi ya ayrı bir akşam oturumunda ya da sonraki uygulamanın ilk yarım saatinde olacak. Duyuru yapılacak.

Kısa Tekrar Notları

- Süreç: çalışmakta olan bir programın o anki görüntüsüdür. Bellekteki hâli, işlemcideki durumu, kaynakları ve durumu (çalışıyor, kesildi, sonlandı) birlikte süreci oluşturur.
- İşletim sistemi süreçleri süreç tablosunda tutar. Her satır bir süreç kontrol bloğudur:
  - kimlik bilgileri
  - durum
  - argümanlar
  - ortam değişkenleri
  - sinyaller

    - açık dosyalar
    - kod ve veri bölgeleri
  - `fork()` dönüş değeri:
    - ebeveynde çocuğun PID'si
    - çocukta 0
    - hata olursa -1
- `exec` ailesi (`execl`, `execv`, `execvp` vb.) çağıran sürecin içeriğini yeni bir programla değiştirir. Yeni süreç oluşturmaz.
- `exit(0)`: normal sonlanma. 0 dışındaki değerler hata ya da özel durum bildirir.
- `atexit(fonksiyon)`: süreç normal sonlanırken çağrılacak bir fonksiyon kaydeder.
- `wait(&durum)`: çocuklardan biri sonlanana kadar bekler. `waitpid(pid, ...)`: belli bir çocuğu bekler.
- `WIFEXITED(durum)`: çocuk normal sonlandı mı? `WEXITSTATUS(durum)`: çocuğun çıkış değeri.
- Zombi süreç: sonlanmış, ama ebeveyni `wait` ile çıkış durumunu almadığı için süreç tablosunda yer tutan süreç. `ps aux` çıktısında durum sütununda `Z` görünür.
- Öksüz süreç: ebeveyni kendisinden önce sonlanmış süreç.
- `pipe(fd)`: `fd[0]` okuma ucu, `fd[1]` yazma ucu. Hata olursa -1 döner.
- Adlandırılmış boru (FIFO): aralarında ebeveyn-çocuk ilişkisi olmayan süreçler arasında da iletişim sağlar.

Detaylı Açıklamalar

`fork` örnekleri. İlk örnekte `fork()`'tan önce bir mesaj, sonra bir mesaj yazılır. "Before fork" bir kez, "after fork" iki kez (ebeveyn ve çocuktan) basılır.

İkinci örnekte ekran çıktısı şöyleydi:

```
process id : 4917
process id : 4918 - result : 0
process id : 4917 - result : 4918
```

İlk satır `fork()`'tan önce ebeveynde yazılır. Çocukta (PID 4918) `fork()` sonucu 0'dır. Ebeveynde (PID 4917) sonuç çocuğun PID'si olan 4918'dir.

Dördüncü örnek (`Ex_4_fork4.c`) ekran karesinden alındı:

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    if (fork()) {
        if (!fork()) {
            fork();
            printf("1 \n");
        } else {
            printf("2 \n");
        }
    } else {
        printf("3 \n");
    }
    printf("4 \n");
    return 0;
}
```

Bu programda toplam 4 süreç vardır: - Ana süreç (P): ilk `fork()` sıfır dışında döndüğü için içteki `if`'e girer ve ikinci kez `fork()` çağırır. Kendisi `else`'e düşer ve "2", sonra "4" yazar. - P'nin ilk çocuğu (C1): ilk `fork()` 0 döndüğü için dıştaki `else`'e düşer ve "3", sonra "4" yazar. - P'nin ikinci çocuğu (C2): `!fork()` doğru olduğu için içeri girer ve bir kez daha `fork()` çağırır. "1" ve "4" yazar. - C2'nin çocuğu (C3): C2 ile aynı yerden devam eder ve "1" ve "4" yazar.

Sonuçta "1" iki kez, "2" bir kez, "3" bir kez, "4" dört kez yazılır. Satırların sırası işletim sisteminin zamanlamasına bağlıdır. Derste görülen sıra `2 4 3 4 1 4 1 4` idi.

`exec` örneği. Ekran karesinden alınan kod:

```
int pid;
printf("Parent does stuff and then calls fork...\n");
pid = fork();
if (pid) {                      // ebeveyn
    // wait(pid);
    printf("... parent do something completely different\n");
} else {                        // çocuk
    printf("Child runs an executable...\n");
    execl("/bin/ls", "/bin/ls", "-l", "/home/turkkol/Desktop/", NULL);
}
exit(0);
```

Çocuk `execl` çağırınca içeriği `ls -l` programıyla değişir ve masaüstündeki dosyaları listeler. Ebeveyn beklemediği için kendi mesajını hemen yazar. Ebeveynin önce çocuğun bitmesini beklemesi isteniyorsa `wait` satırı açılır. Doğru kullanımı `waitpid(pid, NULL, 0)` ya da `wait(NULL)`'dır.

Zombi ve öksüz süreçler. Çocuk sonlanınca çıkış durumunu ebeveynine bildirir. Ebeveyn bunu `wait` ile alınca işletim sistemi çocuğu süreç tablosundan siler. Ebeveyn `wait` çağırmazsa (örneğin sonsuz döngüdeyse) çocuk zombi olarak kalır. Ebeveyn çocuktan önce sonlanırsa çocuk öksüz kalır.

Boru örneği. Ekran karesinden alınan kod (`Ex_1_pipe1.c`):

```
int fd[2];              // fd[0] okuma ucu, fd[1] yazma ucu
if (pipe(fd) == -1) {
    printf("An error occured with oppening pipe");
    return 1;
}
int childpid = fork();
if (childpid == 0) {    // çocuk: yazar
    close(fd[0]);
    int x;
    printf("Enter a number: ");
    scanf("%d", &x);
    write(fd[1], &x, sizeof(int));
    close(fd[1]);
} else {                // ebeveyn: okur
    int y;
    close(fd[1]);
    read(fd[0], &y, sizeof(int));
    close(fd[0]);
    printf("Child with PID: %d sent %d \n", childpid, y);
}
```

Derste 5 girildi. Ebeveyn "Child with PID: … sent 5" yazdı.

İkinci boru örneğinde istemci (ebeveyn) kullanıcıdan bir dosya adı alıp boruya yazar. Sunucu (çocuk) bu adı okur, dosyayı açar ve içeriğini aktarır. Bu örnek ekran paylaşımı koptuğu için tamamlanamadı.

Ders sonrası notlar

- Derste `exec` için "child process'in kendi kendine bir şeyler yapma hakkı var" ve "bu child bunun içinde bir tane daha child oluşturmuş olacak, ls'i çalıştıran bir çocuk oluyor" dendi. Bu yanlış. `exec` yeni süreç oluşturmaz. Çağıran sürecin kod ve verisini yeni programla değiştirir ve PID aynı kalır. Örnekte `ls`'i çalıştıran süreç `fork` ile oluşturulan çocuğun kendisidir. `execl` başarılı olursa sonraki satırlar (`exit(0)`) çocukta hiç çalışmaz. Kod `execl` sonrasına bir `printf` eklenip derlenerek denendi: o satır hiç basılmadı.
- Derste `wait(NULL)` için "bütün çocuklar bitmeden ebeveyn bitmez" dendi. Bu yanlış. `wait(NULL)` yalnızca herhangi bir çocuk sonlanana kadar bekler; slaytta da "one of its children" yazıyor. Bütün çocukları beklemek için `while (wait(NULL) > 0);` döngüsü kullanılır. Koddaki `wait(pid)` de doğru bir çağrı değildir: `wait`'in parametresi PID değil,

çıkış durumunun yazılacağı bir `int *`'dır. Örnek `waitpid(pid, NULL, 0)` ile derlenip çalıştırıldı; ebeveynin mesajı `ls` çıktısından sonra geldi.

- Derste `atexit` için "çocuk süreç bittiği zaman parent cleaner'ı çağıracağız" dendi. Doğrusu: `atexit` ile kaydedilen fonksiyon, onu kaydeden süreç normal sonlanırken (`exit` ya da `main`'den `return`) çağrılır. Çocuğun sonlanmasıyla ilgisi yoktur. Kayıt `fork`'tan önce yapılırsa çocuk da bu kaydı devralır. Denemede temizleyici fonksiyon hem çocukta hem ebeveynde, her biri kendi PID'siyle bir kez çalıştı. Çocuk `exit(3)` ile çıkınca ebeveynde `WIFEXITED` doğru, `WEXITSTATUS` 3 çıktı; yani sıfır dışı çıkış değeri de normal sonlanmadır.
- Zombi süreç `kill -9` ile öldürülemez, çünkü zaten sonlanmıştır. Çözüm ebeveynin `wait` çağırmasıdır ya da ebeveyni sonlandırmaktır. Ebeveyn sonlanınca zombi `init/systemd` sürecine geçer ve o tarafından temizlenir. Derste "ebeveyni terminate ederseniz yine zombi ortaya çıkıyor" denip zombi ile öksüz karıştırıldı. Küçük programlarla denendi: `wait` çağırmadan uyuyan ebeveynin çocuğu `ps` çıktısında `Z` durumunda göründü; ebeveyni önce sonlanan çocukta `getppid()` başka bir PID gösterdi.
- POSIX yalnızca "sinyallerle ilgili bir standart" değildir. POSIX (Portable Operating System Interface), UNIX benzeri işletim sistemlerinin programlama arayüzünü tanımlayan geniş bir standartlar ailesidir. Sinyaller, borular, `fork`, Pthreads bunun parçasıdır.
- Altyazıdaki "has duplex" ifadesi "half-duplex" demektir. Sıradan (anonim) boru tek yönlüdür. İki yönlü iletişim için iki boru gerekir.
- Derste `getpid`'in 0 olduğu söylendi, hoca hemen düzeltti. Çocukta 0 olan `getpid()` değil, `fork()`'un dönüş değeridir.
- Boru örneğinde `#include <unistd.h>` eksik (`pipe`, `fork`, `read`, `write`, `close` için gerekir). GCC 15 bu eksik bildirimi uyarı değil hata olarak verdi. Başlık eklenip `gcc -Wall` ile derlenen program, girdi 5 verilince "Child with PID: … sent 5" yazdı.
- `fork4` programı derlenip çalıştırıldı. "1" iki kez, "2" ve "3" birer kez, "4" dört kez yazıldı. Derste anlatılan sonuçla aynı.
- Derste süreç ağacı "büyükbaba, amca, amca çocuğu" benzetmesiyle anlatıldı ve karışıklık oldu. Doğru ağaç yukarıda: P'nin iki çocuğu var (C1 ve C2), C2'nin bir çocuğu var (C3).
- Anlatılamayan FIFO ve sinyal konuları için `BLM3011_Uygulama_3.pdf` slaytlarına bakın. Sonraki uygulamanın başında telafi yapılacağı söylendi.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.